



UNIVERSITY OF BERGEN

Project 2

Object Detection

INF265 Deep Learning

Andreas Wold Sløgedal
Markus Dybevold Simonsen

Spring | 2026

Table of Contents

1	Object Localization	2
1.1	Problem Setup & Data Exploration	2
1.1.1	Problem Formulation	2
1.1.2	Dataset & Preprocessing	2
1.1.3	Explanatory Analysis	3
1.2	Model Design, Training & Hyperparameter Selection	5
1.2.1	Model Architecture	5
1.2.2	Loss Function	6
1.2.3	Performance Metrics	7
1.2.4	Training Procedure	8
1.2.5	Hyperparameter Selection & Tuning	9
1.3	Model Selection & Performance	10
1.4	Results & Discussion	11
1.4.1	Results & Interpretations	11
1.4.2	Strenghts & Limitations	15
1.4.3	Possible Improvements	15
2	Object Detection	16
2.1	Approach & Design	16
2.1.1	Problem Formulation	16
2.1.2	Data Preprocessing & Analysis	16
2.1.3	Grid Representation	17
2.1.4	Loss Function	17
2.1.5	Performance Metrics	18
2.2	Models & Hyperparameter Choices	18
2.2.1	Baseline Architecture	18
2.2.2	Improved Models	19
2.2.3	Hyperparameters	19
2.3	Model Selection & Performance	20
2.3.1	Validation Comparison & Model Selection	20
2.3.2	Final evaluation on test set	21
2.4	Result and Visualizations	21
2.4.1	Training Curves	21
2.4.2	Loss Component Analysis	22
2.4.3	Detection Examples	22
2.5	Discussion and Conclusion	23
2.5.1	Interpretation of results	23
2.5.2	Strengths and limitations	24
2.5.3	Possible improvements	24
	Division of labor	25
3	Table of Figures	26
4	Bibliography	26
5	Note on use of Artificial Intelligence	26

1 Object Localization

1.1 Problem Setup & Data Exploration

1.1.1 Problem Formulation

Object localization is a fundamental task in deep learning that involves drawing a bounding box around an object in an image while simultaneously classifying it. In this project, we address an object localization problem using an augmented version of the MNIST dataset. In this dataset, digits are randomly positioned, scaled, and rotated, with additional noise applied to the background. The objective is to design, train, and evaluate convolutional neural networks capable of jointly performing three tasks: detecting the presence of an object, classifying it, and accurately predicting the corresponding bounding box coordinates.

1.1.2 Dataset & Preprocessing

Before training the localization model, it is necessary to ensure that the training, validation, and test datasets are properly loaded and correctly formatted. After loading the data, all images were converted to floating-point tensors and reshaped to size $(1 \times 48 \times 60)$, where the dimension 1 represents the grayscale channel, and 48×60 correspond to the image height and width. The channel dimension was added when missing to ensure compatibility with convolutional neural networks, which expect inputs of shape (C, H, W) . In addition, a target tensor representing the object is loaded on the following form.

$$[pc, x, y, w, h, c]$$

Where pc is a boolean that represents whether the image is present or not. (x, y) represents the normalized coordinates of the center of the bounding box, w and h represent the width and height of the bounding box, and c is the class label. The coordinates are defined in the coordinate system of the image, where $(0,0)$ represents the upper left corner of the image, and $(1,1)$ the bottom right corner.

In addition, the input images were normalized prior to training. This step improves the training process by ensuring that gradients behave more consistently and by preventing very small or very large pixel values from dominating the optimization. Although the images are grayscale, normalization remains beneficial, as the pixel values are not inherently standardized. By centering the data and scaling it to unit variance, the model can focus more effectively on relevant spatial patterns rather than absolute intensity values. The normalization was performed using the mean and standard deviation computed exclusively from the training set, as shown in the equation below, and the same statistics were subsequently applied to the validation and test sets to avoid data leakage. A small constant, 10^{-8} , were added to the denominator to avoid division by zero and ensure numerical stability without changing the result meaningfully.

$$X_{norm} = \frac{X - \mu_{train}}{\sigma_{train} + 10^{-8}} \quad 1$$

Furthermore, a series of sanity checks were performed to ensure the correctness and consistency of the dataset before training. These checks concluded that the input tensors had the expected shape $(N, 1, 48, 60)$ and that the targets followed the required $(N, 6)$ format. No

missing or invalid values were detected in either the inputs or targets. The object presence indicator pc was confirmed to be strictly binary, and class labels were verified to be integer-valued and within the valid range $[0, 9]$. Furthermore, for samples where an object is present, all bounding boxes were found to have positive width and height and to lie within the image boundaries. The proportion of images containing an object was also consistent across splits.

1.1.3 Explanatory Analysis

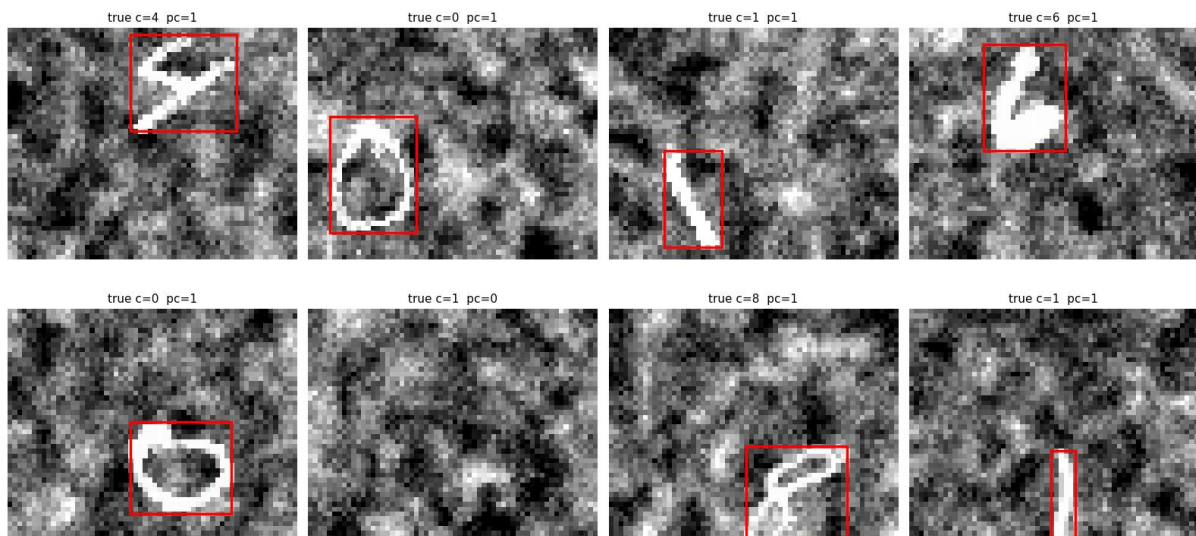


Figure 1 - Example pictures with true bounding boxes.

Figure 1 shows a set of example pictures from the training set, along with their true class c , whether they are present, and their corresponding bounding boxes. By inspecting the images, we can identify several challenges present in this classification task. First, the presence of strong background noise makes it difficult to clearly distinguish the digits from the background, which increases the difficulty of both identifying the correct digits in addition to precisely determining the bounding box. Additionally, some images do not contain any digit (i.e., $pc = 0$), introducing an extra challenge, as the model must learn to both localize and classify the digits, but also correctly identify when no object is present.

Furthermore, the dataset appears to be well balanced as seen in Figure 2. All classes occur approximately 5,000 to 6,000 times, with no class exhibiting a significant imbalance. In addition, the number of images without any object is of the same order of magnitude as each individual digit class, indicating that the detection component of the task is also balanced. This suggests that the model is unlikely to suffer from bias toward any particular digit due to class imbalance.

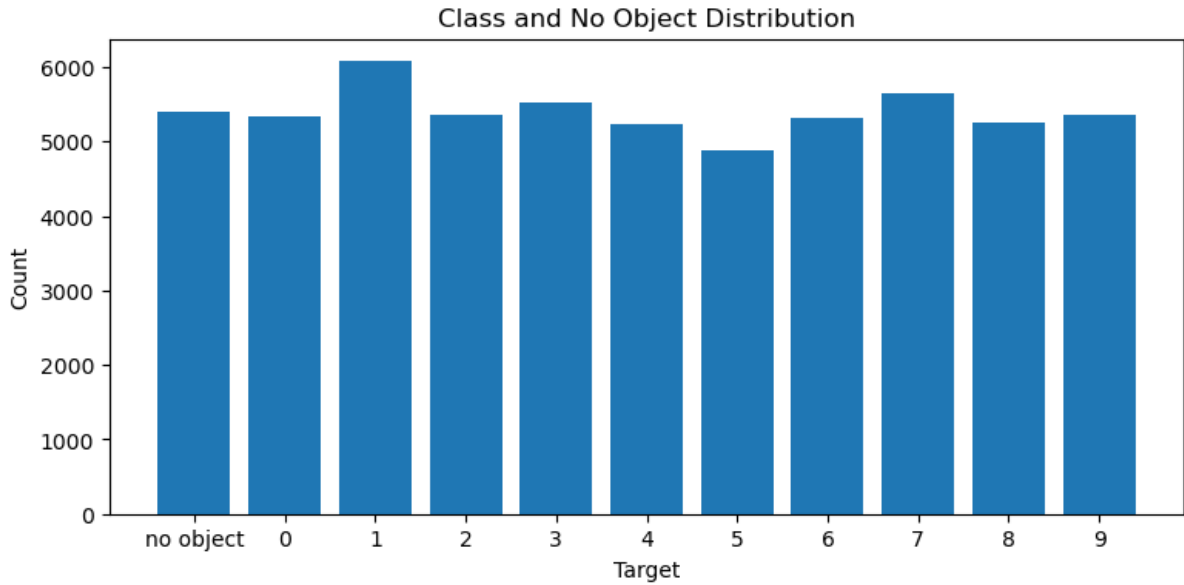


Figure 2 - Class and no object distribution.

Inspection of the target bounding boxes shown in Figure 3 and Figure 4 suggests that digit 1 tends to have a smaller width than the other classes. However, this reflects a genuine property of the data rather than an imbalance that requires correction. Since the dataset also includes random transformations such as rotation and resizing, the model is exposed to a range of box shapes and sizes during training, which helps improve its robustness. Hence, performing any specific adjustment for this class would likely be unnecessary. Nevertheless, if the model were to significantly underperform on this digit, targeted measures could be considered. For the other digits, no particular irregularities were observed, and their bounding box characteristics appear consistent across the dataset.

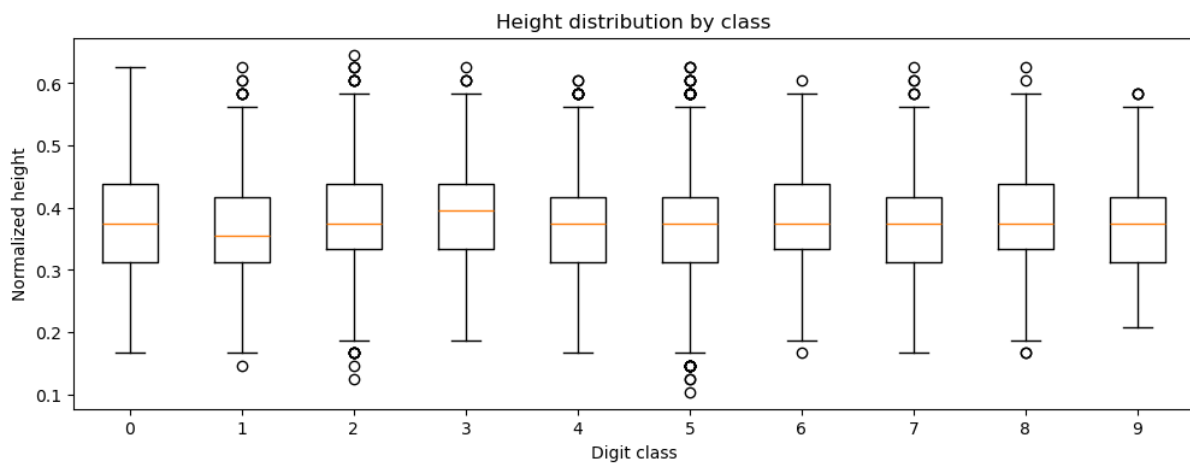


Figure 3 - Height of bounding boxes by class.

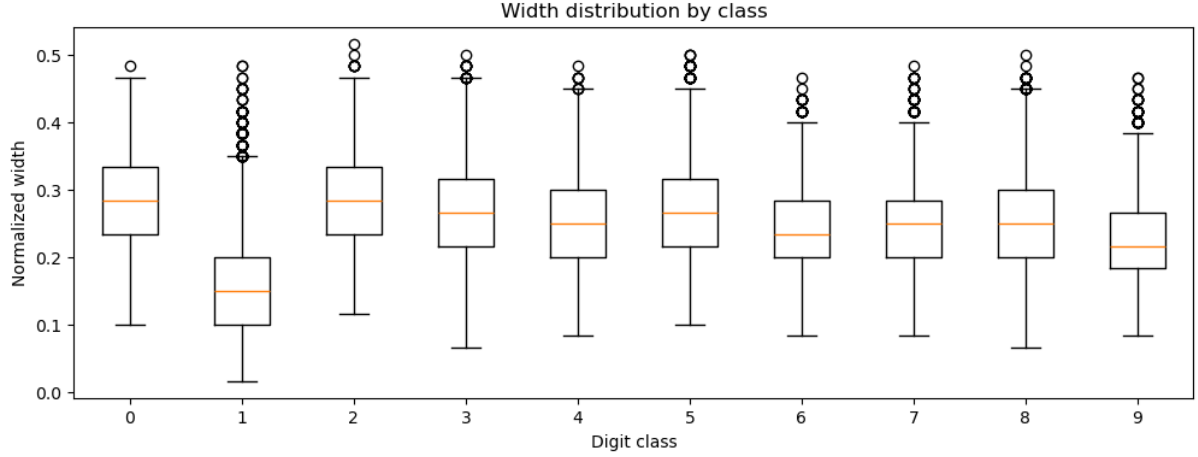


Figure 4 - Width of bounding boxes by class.

Additional exploratory analyses, including the spatial distribution of object centers and the distribution of bounding box aspect ratios, were conducted. However, these analyses did not reveal any significant irregularities or patterns that warranted further adjustments to the modeling approach.

1.2 Model Design, Training & Hyperparameter Selection

1.2.1 Model Architecture

To address the object localization task, three convolutional neural networks (CNN) architectures were implemented. CNNs are well-suited for this problem due to their ability to extract features from images. Which for the task of classification and localization is essential to achieve a sufficient result. All models take the greyscale images of shape $(1, 48, 60)$ as input, and outputs a vector of size $C + 5$, where $C = 10$ is the number of digit classes. The final output therefore has dimension 15 and represents the predicted objectness score, bounding box parameters and class logits corresponding to.

$$[pc, x, y, w, h, c_1, \dots, c_{10}]$$

The Baseline Architecture – LocalizerV1

Before testing more complex detection models, we defined a baseline architecture. A baseline model is useful because it creates a reference point for evaluating the performance of the later models. Instead of aiming for optimal performance, the baseline model has a basic architecture which can help us understand whether hyperparameter tuning or architectural reconstruction will benefit performance the most. It also helps verify that the preprocessing, pipeline and loss function was implemented correctly.

The architecture begins with a convolutional layer with 8 output channels, a 3×3 kernel, and padding 1. This preserves the spatial resolution of the input while extracting low-level local features such as edges and simple digit contours. A ReLU activation is then applied to introduce non-linearity. The resulting feature maps are downsampled using 2×2 max pooling with stride 2, reducing the spatial dimensions from 48×60 to 24×30 . This decreases computational costs while increasing the effective receptive field of subsequent layers.

To obtain a fixed-size representation before the fully connected layers, the network uses adaptive average pooling to map the feature maps to size $(4, 4)$. Since the tensor at this stage has shape $(8, 4, 4)$, flattening produces a feature vector of length $8 \times 4 \times 4 = 128$. This vector is passed through a first fully connected layer with 32 hidden units and ReLU activation, followed by a final linear layer that maps the hidden representation to the 15 output values.

Improved Deeper Model – LocalizerV2

The second architecture was designed as a deeper and more expressive extension of the baseline model. This model stacks three convolutional layers with an increasing number of channels. This enables the model to learn more complex and hierarchical visual features. The network applies the sequence $1 \rightarrow 16 \rightarrow 32 \rightarrow 64$ channels through 3×3 convolutions with padding 1. This ensures that each convolution preserves its spatial dimensions before pooling. Two 2×2 max-pooling operations reduce the feature map size from 48×60 to 24×30 and then to 12×15 . This deeper structure increases the receptive field and enables the network to learn more abstract hierarchical features than the baseline model. After the convolutional blocks, adaptive average pooling maps the tensors to shape $(64, 4, 4)$, which is flattened to a 1024-dimensional representation. This step ensures a consistent feature vector size regardless of the preceding spatial dimensions, making it compatible with the fully connected layers while also summarizing spatial information across the feature maps. The prediction head consists of a fully connected layer with 128 hidden units and ReLU activation, followed by a final linear layer of size $C + 5$, as in the baseline model.

Improved Deeper Model with Batch Normalization & Dropout – LocalizerV3

The third localization model retains the same convolutional structure as the second model, including the channel progression, pooling operations, adaptive average pooling, and prediction head dimensions. The main difference lies in the inclusion of batch normalization after each convolutional layer and dropout in the fully connected head. Batch normalization is used to stabilize the distribution of intermediate activations, which improves numerical stability and facilitates optimization during training. In addition, a dropout layer with probability $p = 0.3$ is applied after the first fully connected layer. During training, dropout randomly deactivates a fraction of the neurons, which acts as a regularization mechanism by reducing co-adaptation between features and helping to prevent overfitting. These additions aim to improve the robustness and generalization performance of the model compared to the previous architecture.

1.2.2 Loss Function

The objective of the training of the localization model is implemented through a custom loss function. This function combines elements from detection, localization and classification. Given a prediction vector of size $C + 5$, as described, the model outputs an objectness score, bounding box parameters, and class logits. Consequently, the loss is decomposed into three terms reflecting these tasks.

The first element is the detection loss L_A , which evaluates whether an object is present in the image. The model outputs a logit, which is transformed into a probability using the sigmoid function

$$\sigma(z) = \frac{1}{1 + e^{-z}}. \quad 2$$

The detection loss is then computed using binary cross-entropy

$$L_A = -[pc \log(\sigma(z)) + (1 - pc) \log(1 - \sigma(z))], \quad 3$$

where $pc \in \{0, 1\}$ denotes whether an object is present.

The second component is the localization loss, L_B . This measures the error between the predicted and the rightful true bounding box parameters. This is done using mean squared error. For a single sample, i , where an object is present this loss will correspond to.

$$L_B^{(i)} = \frac{1}{4}[(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2 + (w_i - \hat{w}_i)^2 + (h_i - \hat{h}_i)^2]. \quad 4$$

To ensure valid predictions, a sigmoid function is applied to the bounding box outputs.

$$\hat{x}_i, \hat{y}_i, \hat{w}_i, \hat{h}_i \in [0, 1], \quad \forall i.$$

Also, since the loss is only meaningful when an object is present, it is applied conditionally using an indicator function

$$L_B = \frac{1}{N} \sum_{i=1}^N \mathbf{1}_{\{pc^{(i)}=1\}} * L_B^{(i)}. \quad 5$$

The last loss component is the classification loss L_C . This evaluates the predicted class labels using cross-entropy loss. For each sample where an object is present, this is defined as

$$L_C^{(i)} = -\log\left(\frac{e^{z_{c_i}}}{\sum_{k=1}^C e^{z_k}}\right), \quad 6$$

Where z_k are the predicted class logits, and c_i is the true class label. Here, like in the localization loss, this term only applies to samples when an object is present hence

$$L_C = \frac{1}{N} \sum_{i=1}^N \mathbf{1}_{\{pc^{(i)}=1\}} * L_C^{(i)}. \quad 7$$

The final loss function is then defined as the sum of detection, localization and classification components.

$$L = L_A + L_B + L_C. \quad 8$$

1.2.3 Performance Metrics

To evaluate the performance of the models, two complementary performance metrics used, classification accuracy and intersection over union (IoU). Both of these metrics were used as

neither metric alone would be sufficient to fully assess the model. The classification accuracy was computed only over images where an object was truly present, meaning that $pc = 1$. A prediction is only considered correct if the model both predicts the presence of an object and rightfully classifies the object. Hence, if N_{obj} denotes the numbers of images that contain an object and N_{corr} denotes the number of right predictions. The accuracy is given by

$$Accuracy = \frac{N_{correct}}{N_{obj}}. \quad 9$$

To evaluate the precision of the bounding box, the intersection over union (IoU) was used. This metric measures the overlap between the predicted bounding box and the actual bounding box and is defined as.

$$IoU = \frac{Area(B_{pred} \cap B_{true})}{Area(B_{pred} \cup B_{true})}. \quad 10$$

The IoU is computed only for images in which an object is present. If the model fails to detect an object, the IoU for that sample is set to 0. This ensures that missed detections are penalized. Finally, the overall performance is defined as the mean of the classification accuracy and the IoU

$$Overall \text{ (mean metric)} = 0.5 * (Accuracy + IoU). \quad 11$$

In addition, to control for the models object detection level a version of the mean Average Precision (mAR) metric is presented. However, since this task contains at most one object per image the standard mAR metric is simplified. In this case, each image contributes a single prediction, and therefore the ranking aspect of mAR is less relevant. Thus, a simplified mAP@0.5 metric was used. A prediction is considered correct if the predicted class is correct and the IoU exceeds a threshold of 0.5. This metric was only used to control the models performance, and not to rank the models.

1.2.4 Training Procedure

All of the localization models were trained using the Adam optimizer. The training was performed using mini batches of size 64, for a total of 20 epochs, subject to an early stop function. During each iteration, the input images and corresponding target values are passed through the network to obtain predictions. Thus, the localization loss is computed. Gradients are then calculated using backpropagation. The optimizer then updates the model parameters, and resets gradients before each step. This is important to prevent accumulation across mini-batches, ensuring that gradients are not carried over between batches.

After each epoch, the model was evaluated on the validation set using a separate evaluation loop. The same loss function as before was used to compute the validation loss, but without updating the model parameters. For both the training and validation sets, the reported loss corresponds to the average loss over all samples. This approach allowed for monitoring both training and validation loss throughout the training process, making it easier to assess convergence behavior and detect potential overfitting. The validation set was used solely for

evaluation and not for parameter updates, ensuring a clear separation between training and model selection.

1.2.5 Hyperparameter Selection & Tuning

Hyperparameters are a set of configuration variables specified prior to training a machine learning model. Hyperparameter tuning refers to the process of selecting and optimizing these variables to achieve optimal model performance (Prince, 2025). In this task, several hyperparameter combinations were explored in order to improve model performance. Initially, different configurations of the adaptive average pooling layer were investigated. In particular the sizes (1, 1), (2, 2) and (4, 4). Of these sizes, the configuration (4, 4) produced significantly better validation performance compared to the other alternatives, likely due to the larger spatial representation preserving more information, which is beneficial for both classification and localization. As a consequence, this configuration was chosen for all the subsequent experiments.

During the hyperparameter tuning we investigated how different values of the learning rate and weight decay affected the model performance. The learning rate controls how large updates are made to the model parameters during training. A higher learning rate may speed up learning but may lead to instability. A lower learning rate will work in opposition. The weight decay is a regularization technique that penalizes large weights by adding a constraint to the loss function, helping reduce overfitting. In our experiments, learning rates of 10^{-4} and 10^{-3} were tested. For weight decay, values of 0 and 10^{-4} were tested.

The hyperparameter tuning that proved to be of most importance was the weighting of the bounding box loss in the overall loss function. Recall Equation 8 from section 1.2.2 where the total loss is defined as the sum of the detection, localization and classification loss. However, here the relative importance of each component can be adjusted through a weighting factor. This is important as the three loss components operate on different scales and represent different objectives. If the localization loss is underweighted the model may not focus enough on this component, leading to poor bounding box predictions. Similarly, if the loss is overweighted the model may prioritize this on expense of detection and classification. Hence, a weighting parameter λ was introduced to investigate this relationship. In particular $\lambda \in \{1, 10, 20\}$ was investigated, corresponding to the modified loss function

$$L = L_A + \lambda L_B + L_C. \quad 12$$

The weighting was applied specifically to the localization term, as it differs fundamentally from the detection and classification losses in that it constitutes a continuous regression problem, whereas detection and classification involve discrete prediction tasks.

During the hyperparameter tuning, an early stopping mechanism was implemented to prevent overfitting. This was done by terminating training when the validation loss did not improve for three consecutive epochs. This also reduced the computational cost by avoiding unnecessary training once the model performance ceased to increase.

1.3 Model Selection & Performance

To evaluate the different models and configurations, all architectures were trained on all possible combinations of learning rates, weight decay and the bounding box weight λ . For each of the configurations the performance was measured on the mean of the accuracy and the intersection over union (IoU) as described in section 1.2.3

From the training curves we observe that the baseline model (LocalizerV1), shows relative slow convergence in addition to consistently higher loss than the deeper models (LocalizerV2 and LocalizerV3). This showcases the baseline model's limited capacity for this localization task. Both of the deeper models exhibit considerable faster convergence and significantly lower validation loss. This is clearly shown in the figure below. However, it is worth noting the slight increase in validation loss for the deeper models in the later epochs, along with the emerging gap between training and validation loss, as shown in the `Object_localization.ipynb` file. This suggests a tendency toward mild overfitting.

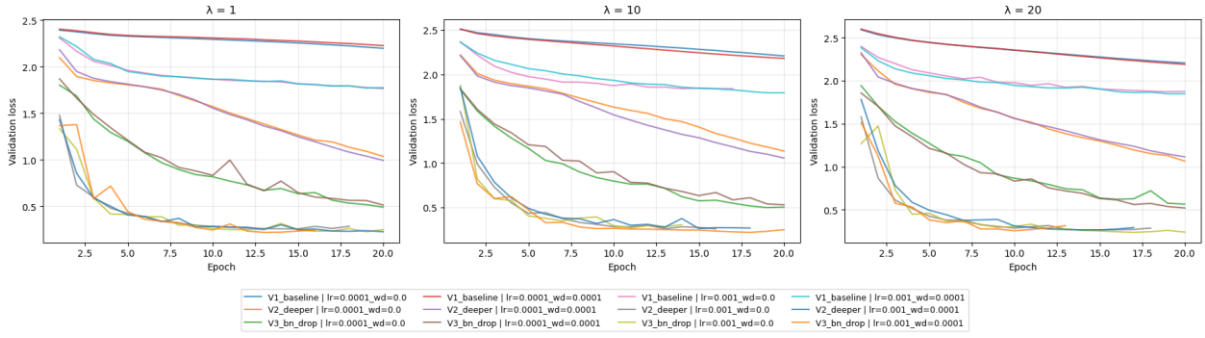


Figure 5 - Model validation loss per. λ .

The effect of the bounding box parameter is substantial as shown in the table below. Increasing the λ clearly increases the IoU indicating better localization performance without affecting the classification accuracy.

λ	LR	WD	Model	Acc	IoU	Overall mAP@.5	λ	LR	WD	Model	Acc	IoU	Overall mAP@.5		
1	0.0001	0.0	V1_baseline	0.1703	0.3705	0.2704	0.0035	1	0.001	0.0	V1_baseline	0.2907	0.4629	0.3768	0.0214
1	0.0001	0.0	V2_deeper	0.6227	0.5561	0.5894	0.2567	1	0.001	0.0	V2_deeper	0.9153	0.5806	0.7479	0.5334
1	0.0001	0.0	V3_bn_drop	0.8312	0.5180	0.6746	0.3635	1	0.001	0.0	V3_bn_drop	0.9287	0.4615	0.6951	0.2421
1	0.0001	0.0001	V1_baseline	0.1610	0.3763	0.2687	0.0040	1	0.001	0.0001	V1_baseline	0.2878	0.4296	0.3587	0.0165
1	0.0001	0.0001	V2_deeper	0.6405	0.5696	0.6051	0.2884	1	0.001	0.0001	V2_deeper	0.9263	0.5906	0.7585	0.5833
1	0.0001	0.0001	V3_bn_drop	0.8162	0.5971	0.7066	0.5520	1	0.001	0.0001	V3_bn_drop	0.9255	0.5721	0.7488	0.5688
10	0.0001	0.0	V1_baseline	0.1817	0.4682	0.3249	0.0077	10	0.001	0.0	V1_baseline	0.2693	0.5134	0.3913	0.0301
10	0.0001	0.0	V2_deeper	0.5968	0.6391	0.6180	0.3538	10	0.001	0.0	V2_deeper	0.9167	0.6808	0.7987	0.7856
10	0.0001	0.0	V3_bn_drop	0.8305	0.6396	0.7350	0.6568	10	0.001	0.0	V3_bn_drop	0.9175	0.5566	0.7370	0.5413
10	0.0001	0.0001	V1_baseline	0.1718	0.4764	0.3241	0.0083	10	0.001	0.0001	V1_baseline	0.2875	0.5166	0.4021	0.0361
10	0.0001	0.0001	V2_deeper	0.6223	0.6506	0.6365	0.3985	10	0.001	0.0001	V2_deeper	0.9117	0.6995	0.8056	0.8359
10	0.0001	0.0001	V3_bn_drop	0.8133	0.6602	0.7368	0.6793	10	0.001	0.0001	V3_bn_drop	0.9333	0.6839	0.8086	0.8356
20	0.0001	0.0	V1_baseline	0.1873	0.4911	0.3392	0.0123	20	0.001	0.0	V1_baseline	0.2753	0.5241	0.3997	0.0335
20	0.0001	0.0	V2_deeper	0.6228	0.6691	0.6459	0.4296	20	0.001	0.0	V2_deeper	0.9180	0.7135	0.8157	0.8540

λ	LR	WD	Model	Acc	IoU	Overall mAP@.5	λ	LR	WD	Model	Acc	IoU	Overall mAP@.5		
20	0.0001	0.0	V3_bn_drop	0.8102	0.6610	0.7356	0.6655	20	0.001	0.0	V3_bn_drop	0.9360	0.6442	0.7901	0.7831
20	0.0001	0.0001	V1_baseline	0.1847	0.4965	0.3406	0.0110	20	0.001	0.0001	V1_baseline	0.2793	0.5299	0.4046	0.0379
20	0.0001	0.0001	V2_deeper	0.6058	0.6602	0.6330	0.3966	20	0.001	0.0001	V2_deeper	0.9147	0.7113	0.8130	0.8398
20	0.0001	0.0001	V3_bn_drop	0.8295	0.6782	0.7539	0.7169	20	0.001	0.0001	V3_bn_drop	0.9230	0.6706	0.7968	0.8162

Table 1 - Performance on validation set

Among all of the tested models, the LocalizerV2, achieved the best performance with configuration $\lambda = 20$, learning rate of 10^{-3} and weight decay of 0. The model achieved an accuracy of 0.918 and IoU of 0.713 and overall score of 0.8157 on the validation set. It's mAP@0.5 corresponded to 0.854. This model and configuration show the best balance between classification and localization. Although, the LocalizerV3 tend to have somewhat stronger classification, its IoU is consistently weaker, yielding a lower overall score. The LocalizerV2 model with these configurations was then chosen automatically due to its superior overall score. Thus, this model was used for final evaluation on the test set. Figure 6 and Figure 7 below show a set of example predictions on the training and validation set respectively for the chosen model. With the predicted bounding boxes marked with the dashed lines.

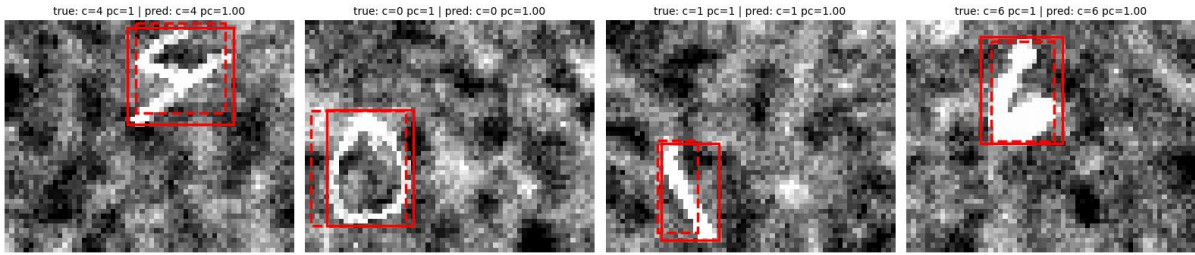


Figure 6 - Model prediction on training set.

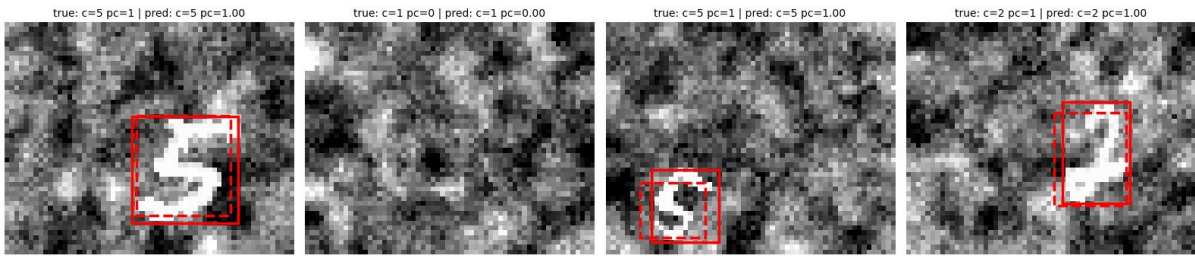


Figure 7 - Model predictions on validation set.

1.4 Results & Discussion

1.4.1 Results & Interpretations

The LocalizerV2 model was tested on the unseen test set to obtain an unbiased estimate of its performance. The performance metrics showed similar results as on the validation set as shown in the table below. Achieving a slightly increased overall performance, and almost an identical mAP@0.5 score.

λ	LR	WD	Model	Acc	IoU	Overall mAP@.5	
1	0.001	0.0	V2_deeper	0.9209	0.7127	0.8168	0.8513

Table 2 - Localization results on test set.

The consistent results across validation and test set indicate that the model generalizes well to unseen data, and that the model avoids overfitting. Figure 8 shows a set of the model predictions on the test set.

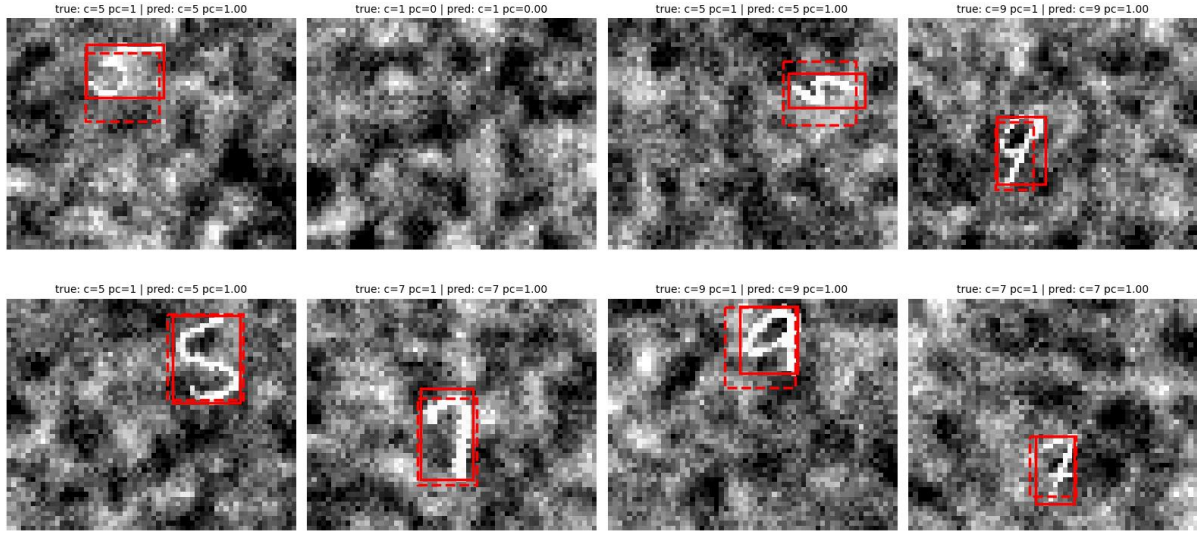


Figure 8 – Localization model predictions on the test set.

The classification accuracy across digits is generally high, ranging from 0.82 to 0.99, as shown in Figure 9. The model achieves highest accuracy on digit 1, indicating that it is the easiest class to recognize. This is most likely because of its distinctive and simple shape.

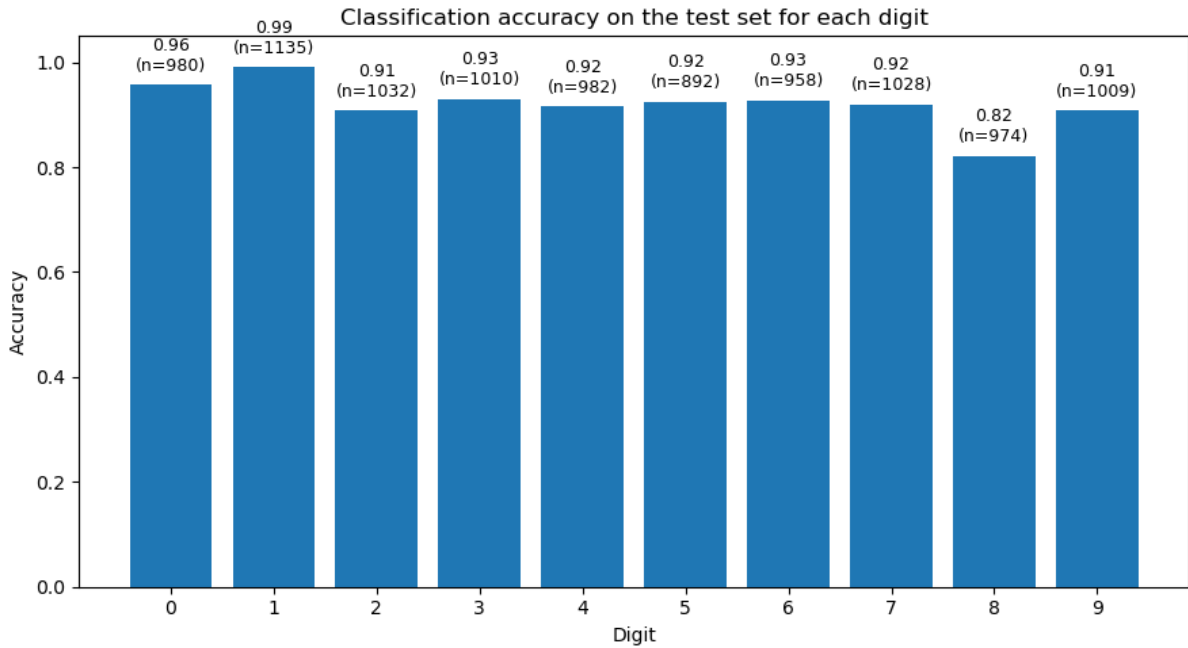


Figure 9 - Classification accuracy per digit.

In contrast, digit 8 shows the lowest accuracy, with an accuracy of only 0.82, suggesting that the model struggles to classify this digit. This is most likely because of its shared similarities with other digits. From the normalized confusion matrix in Figure 10 we can observe that the model struggles with distinguishing digit 8 from 0, 2, 3, 5, 6 and 9.

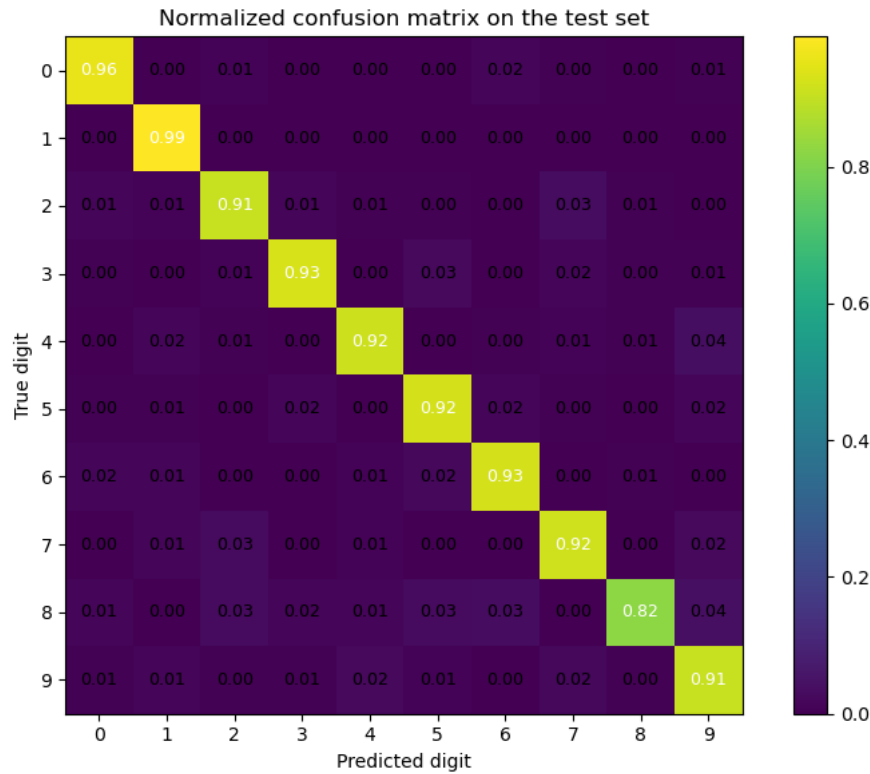


Figure 10 - Normalized confusion matrix.

The model achieves near-perfect presence detection, with an overall accuracy of 0.999. Every single one of the empty images is correctly identified, and only 13 out of the 10,000 images containing an object is missed. This indicates that the model robustly distinguishes between images with and without digits, with negligible false positives and very few false negatives as shown in the confusion matrix below.

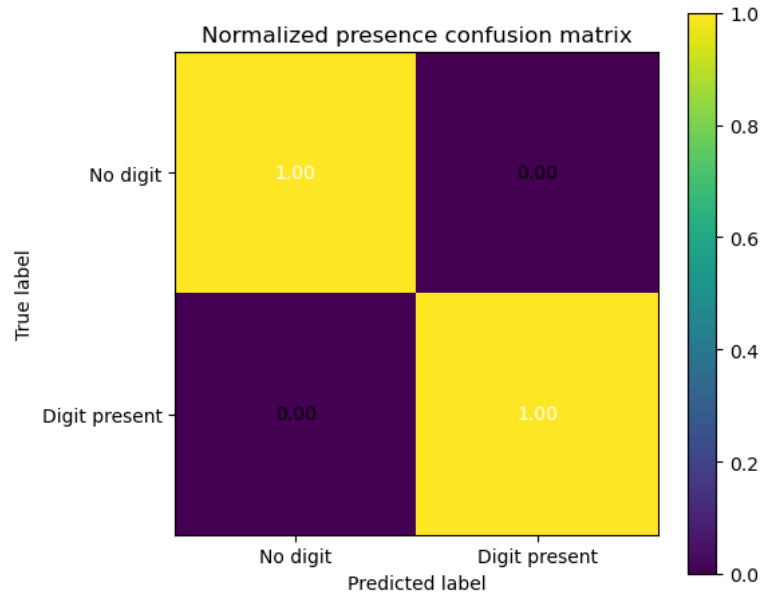


Figure 11 - Normalized presence confusion matrix.

The localization performance is also relatively consistent across most digits, typically ranging between 0.71 and 0.76. Digit 0 achieves the highest IoU with 0.76. This could be interpreted as the model manages to localize well-defined shapes as this digit. In contrast, digit 1 has a clearly noticeable lower IoU of 0.61. This, despite having the highest classification accuracy. This suggest that the model can easily recognize 1, but struggles to accurately predict its bounding box.

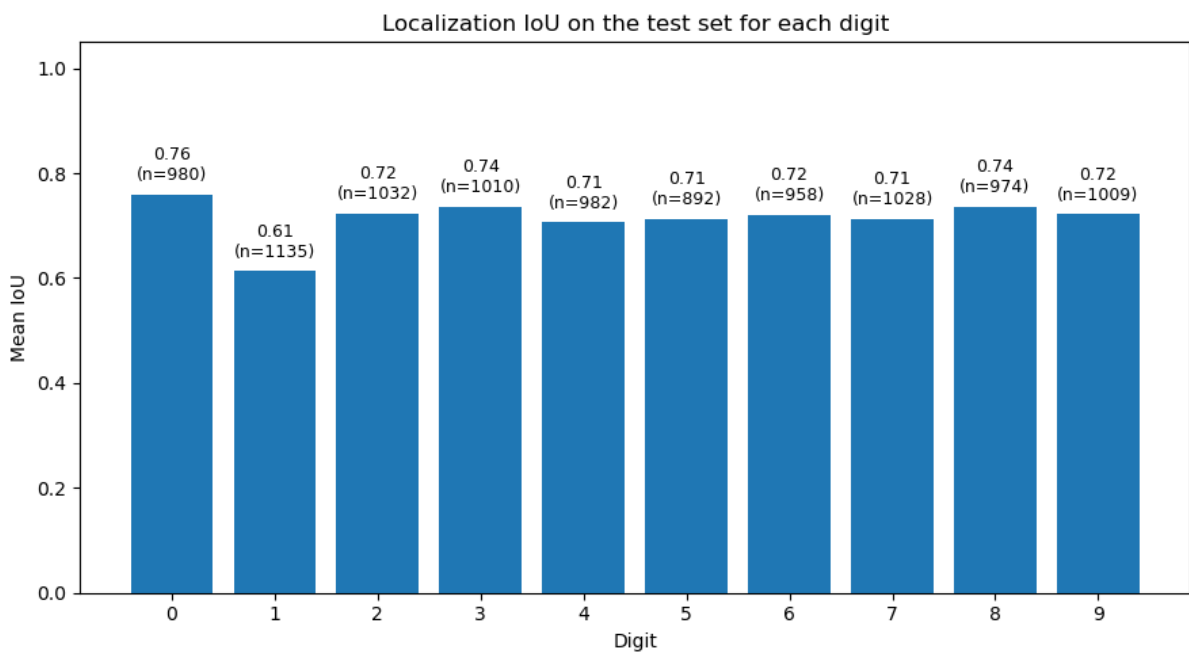


Figure 12 - Localization IoU per digit.

The analysis of the localization center bias displayed in Figure 13 shows that that the horizontal- and vertical errors are generally close to zero, for all digits. Thus, indicates that the model does not indicate any systematic bias towards any direction in its bounding box

predictions. This suggests that the localization errors are mostly random rather than systematic, which is a desirable property.

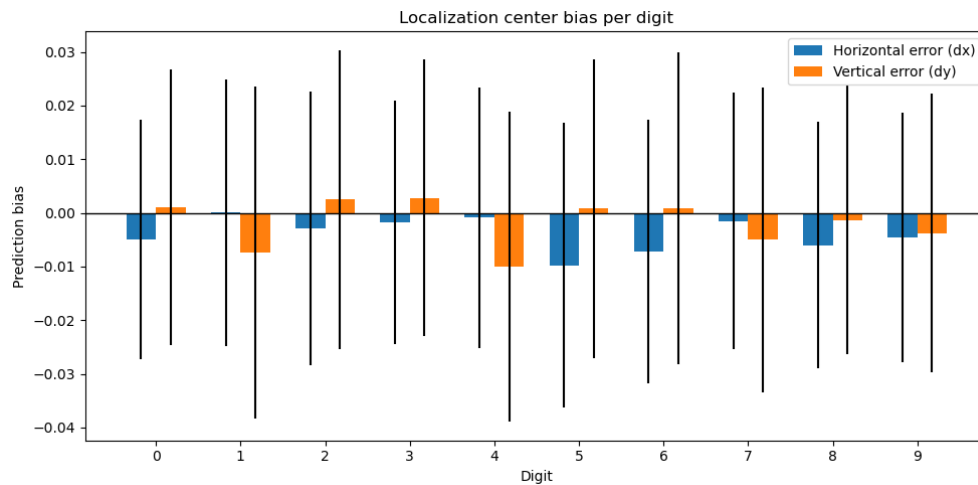


Figure 13 - Localization center bias per digit

From the hyperparameter tuning results, the importance of learning rate and bounding box weighting is highlighted. A higher learning rate of 10^{-3} consistently led to better performance, likely due to faster convergence. Additionally, the increase of the bounding box weight improved the localization performance vastly. Demonstrating the importance of balancing classification and localization objectives.

1.4.2 Strengths & Limitations

Overall the model demonstrates strong and consistent performance across both classification and localization tasks, as evidenced by the similar results on the validation and test set. In addition the model exhibits strong robustness across all the digits classes, and both classification and localization performance remains stable across the datasets. Furthermore, the use of early stopping reduced the computational cost without hurting the performance, resulting in an efficient training process.

However, some limitations must be noted. First and foremost, the model shows reduced localization performance for certain digits. Especially the digit 1. This suggests that the model fails to detect thin and long shapes such as this digit. In addition, some confusion remains between visually similar digits. This indicates that the model could benefit from improving feature representation.

The code implementation follows an object-oriented, iterative and procedural approach. This ensures a robust storing of models, data results and evaluations. Furthermore, the iterative approach makes training and validation and evaluation straightforward to run and debug.

1.4.3 Possible Improvements

Even though the model exhibits strong results, some measures could be taken to further enhance its performance. First and foremost, additional hyperparameter tuning would most likely lead to an improved model. This applies to all of the parameters that were tuned during

the training process. For example, we observe that the model chooses the highest possible bounding box weight 20. This suggests that increasing the weight further could lead to an improved model. However, increasing the weight from 10 to 20 shows clear signs of convergence in the validation performance metrics, indicating that any additional gains are likely to be marginal and therefore not necessary for this project.

Furthermore, tuning additional hyperparameters that have not yet been explored may lead to improved performance. For example, preliminary experiments showed that modifying the adaptive average pooling layer produced different results, suggesting that further investigation of this and similar components could enhance the model's performance. In addition, the loss function currently only adjusts the weight of the bounding box parameter (λ) due to its distinct role. However, optimizing the weights of the other components may also yield further improvements.

The data utilized in this project was already augmented, meaning its objects were slightly rotated, resized, and noise was added to the background. Further data augmentation may also yield improved generalization and robustness, potentially enhancing the model's performance on unseen data. Another possible improvement would be to explore other model architecture and design choices beyond the ones considered in this project. The models implemented here mainly differs in depth and number of channels, in addition to the inclusion of batch normalization and dropout. While these modifications improve the performance, there may be beneficial to explore other model architectures.

2 Object Detection

2.1 Approach & Design

2.1.1 Problem Formulation

The objective of this object detection task is to identify all digits that are present in the image, as well as predicting their class and bounding boxes. Compared to the object localization task, there can be several objects present per image. Therefore, we will use a grid-based approach where images are divided into a grid of $H_{out} \times W_{out}$ cells. And each cell will predict whether there is an object present, and if there is, estimating its bounding box and class label. The images used have digits that are slightly rotated and resized. The images also contain noise, making it more difficult for the model to learn patterns.

2.1.2 Data Preprocessing & Analysis

The preprocessing pipeline starts with loading the training, validation and test datasets (of size 48×60). The expected outputs are stored as lists of bounding box annotations for each image (`list_y_true_XXX.pt`). Each object is represented as a tensor on the form $[p_c, x, y, w, h, c]$, using the same normalized coordinate system as described in Section 1.2.1.

Inspecting the dataset showed that the detection task is challenging because the images can contain several digits and noisy backgrounds. The grid-based setup also means that many cells are empty, so accuracy alone will not say much about model performance.

Moreover, an outlier pixel value of 1.41 was found in the training set, which is out of the expected range $[0,1]$. This anomaly is likely a result of augmentation artifacts in the dataset creation. All splits were normalized using training set statistics (z-score), preventing data leakage.

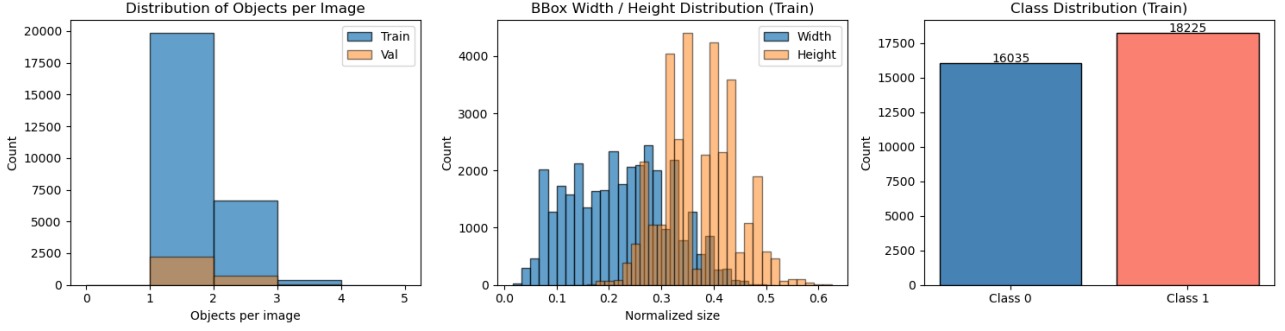


Figure 14 - Dataset (*list_y_true_XXX.pt*) analysis for object detection

2.1.3 Grid Representation

Since the object detection model operates on grid cells, the annotations are converted to grid representation through the `convert_list_to_grid()` function, which generates the grid targets for training. For all images, the bounding boxes are transformed into a tensor of shape $(H_{out}, W_{out}, 6)$. Then the grid cell containing the center of an object is identified, and bounding box parameters are converted to coordinates relative to that grid cell. Ultimately, the tensor represents each object in the local coordinate system of its corresponding cell. In images where there are several objects present in each grid cell, the model keeps the object with the larger bounding box area. To verify the implementation, a sanity check was performed, confirming zero conversion error against the pre-processed grid labels provided in `detection_XXX.pt`.

2.1.4 Loss Function

The training objective used for the detection model is implemented through the function `detection_loss()`. While the loss shares the same components as in the localization task, it is adapted to a grid-based formulation where it is applied to all grid cells rather than a single object per image.

Firstly, objectness loss (LA) evaluates whether a grid cell contains an object. This term is measured using binary cross-entropy and is applied to all grid cells, regardless of whether an object is present in the grid cell or not. The second component is the localization loss (LB), which measures the error between the predicted and ground-truth bounding box parameters (x, y, w, h) . This loss is computed using mean squared error (MSE) and is applied only to the grid cells that contain an object. The final component is the classification loss (LC) and evaluates the predicted class label of detected objects using cross-entropy loss. This component is also only applied to grid cells that contain an object.

Ultimately, the total detection loss is computed as a weighted sum of the three mentioned components, and the loss is summed over all grid cells and averaged over the batch during

training. To make sure the loss implementation works as expected, a sanity check is performed by computing the loss on random model predictions. A baseline model is created through the `build_model()` function, and the output confirms that the initial loss together with the individual contributions L_A , L_B and L_C is finite and well-defined.

2.1.5 Performance Metrics

To evaluate detection performance, several metrics are used. As described in Section 1.2.3, classification accuracy and intersection over union (IoU) measure classification correctness and bounding box overlap, respectively.

However, in the grid-based detection setting, accuracy becomes less informative, as most grid cells do not contain objects. This can lead to artificially high accuracy if the model predicts empty cells correctly, without reflecting true detection performance.

IoU remains a useful measure of localization quality, but it does not capture the full detection task. Therefore, mean Average Precision (mAP) is used as the primary evaluation metric. Unlike grid-based metrics, mAP evaluates predicted bounding boxes and class labels after transforming them back to the global coordinate system, which gives a more comprehensive measure of detection performance.

2.2 Models & Hyperparameter Choices

2.2.1 Baseline Architecture

Before testing more complex detection models, we defined a baseline model which is useful because it creates a reference point for evaluating the performance of the later models. Instead of aiming for optimal performance, the baseline model has a basic architecture which can help us understand whether hyperparameter tuning or architectural reconstruction will benefit detection performance the most. It also helps verify that the preprocessing, pipeline and loss function was implemented correctly.

The baseline model is a convolutional network model designed to produce predictions for each grid cell in the image. The model contains three convolutional layers with kernel size 3×3 and padding 1, followed by ReLU activation functions. Padding equal to 1 ensures that spatial dimensions are preserved after each convolution. The model ensures that feature channels gradually increased from 1 to 16, and 32 then 64. This is a key architectural feature which ensures that the model learns progressively more complex features. However, the model remains fully convolutional and does not use fully connected layers, which preserves the spatial structure of the image and allows the network to produce predictions for each grid cell.

After the convolution, an adaptive pooling layer is used to make the spatial output dimensions match the grid size of $H_{out} \times W_{out} = 2 \times 3$. This ensures the model output has the correct grid structure regardless of the size of the feature map. The final prediction layer is implemented as a 1×1 convolution. This layer maps the 64 feature channels at each grid cell directly to $C + 5$ output values. In this project $C = 2$ because we are only working with

the two digits 0 and 1. Hence, each cell predicts 7 values: one objectness score, four bounding box parameters, and two class scores. The output tensor has shape $(N, C + 5, H_{out}, W_{out})$ before it is permuted to $(N, H_{out}, W_{out}, C + 5)$. This is the format expected by the loss function and the upcoming detection pipeline as well.

2.2.2 Improved Models

In this section we define two alternative models to explore whether increased model capacity and architectural modifications improve detection performance. These models are compared against each other, and the baseline architecture described in the previous section. This comparison will help investigating how deeper feature extraction, normalization techniques, and regularization influence the model’s ability to detect objects.

The first architectural variant (VariantA) uses a deeper convolutional backbone with more feature channels and batch normalization. Compared to the baseline model, the number of channels in this variant increases from 32 to 64, 128 and to 256. Each layer is followed up by batch normalization and a ReLU activation function. Batch normalization improves convergence by stabilizing training through normalizing intermediate activations. Moreover, max pooling layers are introduced between convolutional blocks to gradually reduce spatial resolution. As in the baseline model, this variant uses an adaptive pooling layer to ensure the spatial dimensions match the detection grid size $H_{out} \times W_{out} = 2 \times 3$.

The second architectural variant (VariantB) introduces residual connections and dropout regularization. Residual blocks are used to help the network learn residual mappings, where the input of the block is added to its output. The point of this is to maintain gradient flow through deeper networks and improve training stability. In addition, a dropout layer is applied in the deeper part of the network to reduce overfitting by randomly reactivating feature channels during training. Both variants follow the output structure of the baseline model, leading to $C + 5$ predictions per grid cell. The output is also reshaped to $(N, H_{out}, W_{out}, C + 5)$ for compatibility with the detection loss.

2.2.3 Hyperparameters

To find the best training configuration for each model, a hyperparameter grid search was performed over learning rates ($lr \in \{1e-3, 3e-4\}$) and weight decays ($wd \in \{1e-4, 1e-3\}$). This resulted in 12 total training runs, and each candidate was trained for up to 20 epochs with early stopping (patience = 5). A learning rate scheduler (ReduceLROnPlateau) was initially included but removed, as the scheduler never had the opportunity to reduce the learning rate before training terminated. The hyperparameter configuration with the lowest validation loss was selected per model. We decided not to try further hyperparameter values due to computational costs.

Model	Best learning rate	Best weight decay
Baseline	1e-3	1e-4
Deep_batchnorm	3e-4	1e-3
Residual_dropout	1e-3	1e-4

Table 3 - Best hyperparameters for each model

The search indicated that the `deep_batchnorm` benefits from stronger regularization (higher weight decay of $1e-3$), whilst the baseline and `residual_dropout` model already regularize through dropout. The best performing configuration for each model is kept directly from the search phase without separate retraining.

To further reduce overfitting, we applied label-preserving augmentations during training: random Gaussian noise ($\sigma \sim \text{Uniform}[0, 0.08]$) and brightness scaling (factor $\sim \text{Uniform}[0.85, 1.15]$). Neither of them affected bounding box coordinates. During inference, an object is detected when the predicted objectness score exceeds a threshold of 0.5 ($p_c \geq 0.5$), which is the standard threshold for binary detection decisions.

2.3 Model Selection & Performance

2.3.1 Validation Comparison & Model Selection

For a fair comparison between the model architectures, the three models were trained using the same preprocessing pipeline, loss function and hyperparameter configuration. Model selection is performed automatically based on the performance on the validation set, and the test set remains unseen for final validation.

The validation results show clear improvement from the baseline model to the more complex architectures. The baseline model achieved an accuracy of 0.939, and IoU of 0.475 and a mAP of 0.125. The deeper batch-normalized model (VariantA) improved the localization performance substantially, reaching an IoU of 0.689, a mean metric of 0.836, mAP of 0.410.

The best validation performance was obtained by the `residual_dropout` model (VariantB). This model achieved an accuracy of 0.989, and IoU of 0.765. Furthermore, it produced the highest detection quality measured by mean Average Precision, with a mAP of 0.539 and $\text{mAP}@50 = 0.922$ respectively.

Based on validation results, the `residual_dropout` model (VariantB) performed the best in all metrics. The model selection was based on mAP, as recommended by the project specification, since it evaluated predictions in the global coordinate frame independently of the grid structure. A summary of all model performance metrics can be found in Table 4.

Model	Accuracy	IoU	Mean metric	mAP	mAP@50
Baseline	0.939	0.475	0.707	0.125	0.434
Deep + BatchNorm (VariantA)	0.982	0.689	0.836	0.410	0.846
Residual + Dropout (VariantB)	0.989	0.765	0.877	<u>0.539</u>	0.922

Table 4 - Summary of model performance metrics on the validation set.

The difference between mAP and mAP@50 reflects the stricter localization requirements of mAP. Whilst mAP@50 indicates that the model reliably detects objects, the lower mAP suggests that some predicted bounding boxes are not perfectly aligned with the actual digit location.

2.3.2 Final evaluation on test set

The `residual_dropout` model was tested on the unseen test set to obtain an unbiased estimate of its performance. The performance metrics showed similar results as on the validation set, with an accuracy of 0.988, IoU=0.768, mean metric=0.878, mAP=0.542 and mAP@50=0.922. The consistent results across validation and test set indicate that the model generalizes well to unseen data.

2.4 Result and Visualizations

2.4.1 Training Curves

The training curves show how the models learn across the 20 epochs during optimization. As shown in Figure 15 (right), the baseline model decreases its loss rapidly early in the training process. However, it converges at a relatively high loss.

In contrast, the more complex models reduce training loss much faster in the beginning of training. Both the `deep_batchnorm` model (VariantA) and the `residual_dropout` model (VariantB) reduce the training loss rapidly in the early epochs and continue improving until the loss approaches near-zero. This suggests that the two more complex models have higher capacity to fit the training data.

Early stopping is applied based on validation loss, and training stops when no improvement is seen for 5 consecutive epochs. This leads to the models training for a different number of epochs. The `deep_batchnorm` model stops early after roughly 16 epochs with a best validation loss of 0.384. The `residual_dropout` stops after roughly 14 epochs with a best validation loss of 0.249. This shows that the `residual_dropout` model keeps improving validation performance for a longer period.

The validation loss curves in Figure 15 (left) also highlights the difference between models. The baseline model decreases slowly and stabilizes at a relatively high loss level, indicating underfitting. Both the `deep_batchnorm` and `residual_dropout` model maintain the low validation losses during training, suggesting they generalize better than the other architectures.

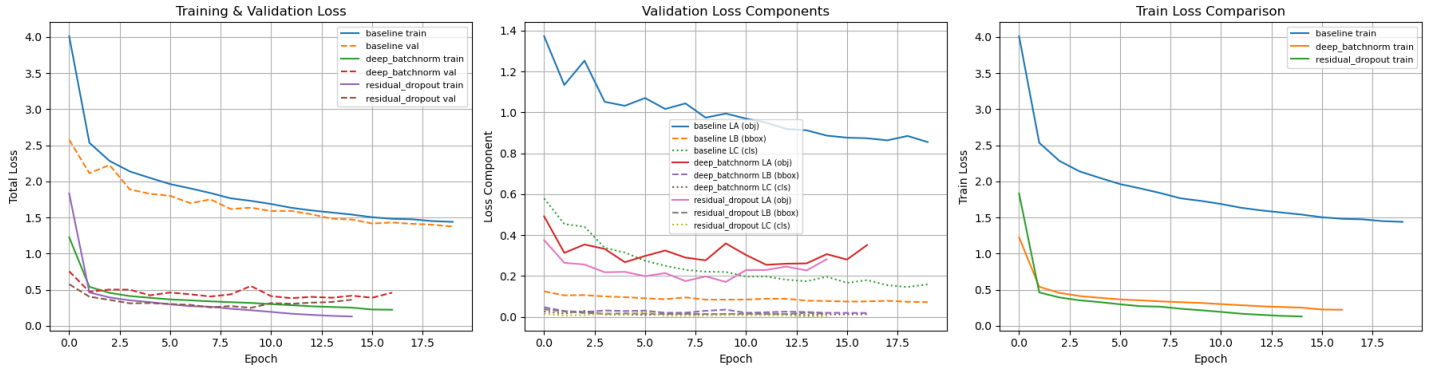


Figure 15 - Loss analysis.

2.4.2 Loss Component Analysis

To better understand the training dynamics, each of the three components of validation loss were analyzed separately. The objectness loss is the most dominant contributor to loss across all models. As seen in Figure 15 (middle), this component decreases slowly for the baseline model and remains relatively high throughout training. This indicates that the baseline model has difficulty predicting whether grid cells contain digits or not.

In contrast, the more complex models reduce objectness loss more quickly during early epochs, converging at a much lower level. The `residual_dropout` model achieves slightly more stable behavior and stabilizes early in training (around epoch 5). This suggests that this model learns to identify grid cells containing objects more consistently.

The bounding box and classification loss are much smaller and stable throughout training. The `residual_dropout` model achieves the lowest bounding box loss, which indicates better localization of the digits that are detected. Overall, the component analysis shows that the architectural improvements enhance objectness prediction, with additional gain in localization accuracy.

2.4.3 Detection Examples

To evaluate the detection performance, example predictions from both the training and validation sets can be seen in Figure 16 and Figure 17 respectively. In the figures, the true bounding boxes are shown as solid lines whilst predicted bounding boxes are shown as dashed lines. The figures show that the `residual_dropout` model can predict the digit classes even in the presence of noisy backgrounds. In most cases, the predicted bounding boxes align with the true ones, indicating accurate localization.

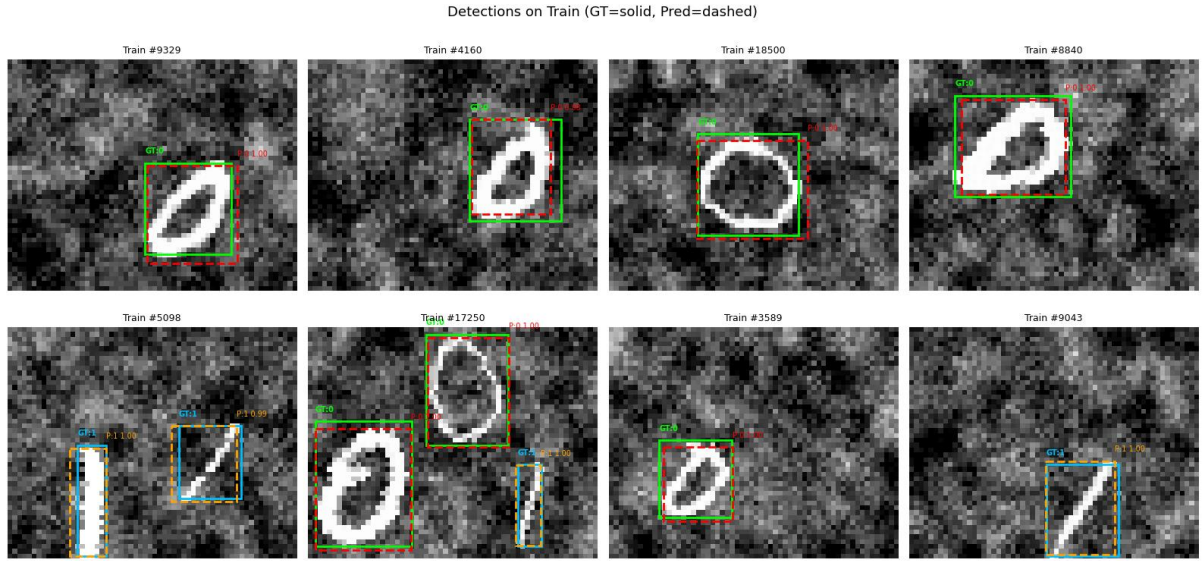


Figure 16 - Detections on the training set.

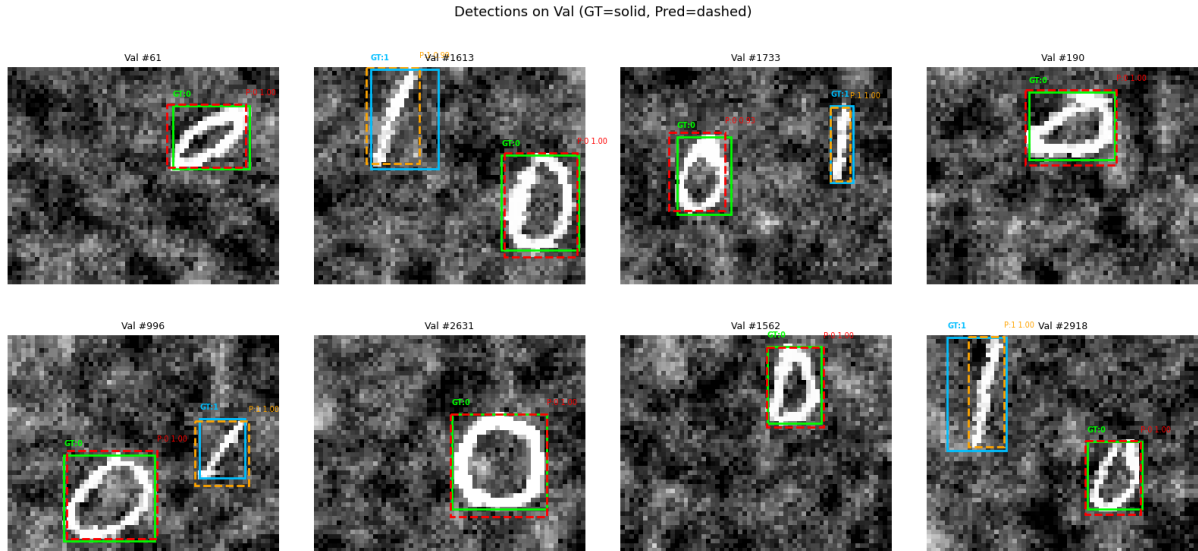


Figure 17 - Detections on the validation set.

Comparing the training and validation examples, the detection quality appears to be consistent across both datasets, which suggests that the model generalizes well and does not overfit. Ultimately, the visual results match the quantitative metrics and confirm the model performs well.

2.5 Discussion and Conclusion

2.5.1 Interpretation of results

The results show that increasing model capacity in terms of depth and introducing architectural improvements such as batch normalization and dropout regularization improves object detection performance. The baseline model was able to detect digits relatively well but

was clearly limited compared to the more complex models. We noticed that the baseline model converged to a relatively high validation loss and produced less accurate bounding boxes.

Both `residual_dropout` and `deep_batchnorm` benefited from deeper feature extraction and batch normalization, which likely stabilized training and allowed the models to learn more effectively. Ultimately, the residual dropout model received the best overall result, gaining the highest scores across all metrics. It is evident that residual connections likely improved gradient flow during training, which benefited the detection accuracy the most. The high performance suggests that the chosen model generalized well to unseen data.

The small gap between validation and test mAP (0.539 vs 0.542) was not surprising given the use of early stopping and dropout regularization, which we specifically used to prevent overfitting. The moderate mAP score is most likely driven by imperfect bounding box localization. mAP@50 of 0.922 indicates that the model reliably detects objects, but that stricter localization (mAP) penalizes predictions where the box overlap falls below the required threshold.

2.5.2 Strengths and limitations

The grid-based architecture allows the model to detect multiple objects within the same image, which is a strength of the detection model. The model also localizes bounding boxes rather accurately despite the background noise and digit rotation. Furthermore, the model is very stable during training. The model learns fast early on in the training process and converges at a relatively low loss value.

However, there are some limitations to the model. The noisy background led to some false positive detections. Sometimes the model picks up random noise as digits, which leads to wrong predictions. In some cases, the predicted bounding boxes visually overlap with the ground truth but are still classified as false positives. The reason for this is likely that metrics rely on an IoU threshold (0.5) to determine whether a prediction is correct. Hence, predictions with insufficient overlap are therefore counted as incorrect even if the visualization looks close to the true bounding box.

All models are implemented using an object-oriented approach. This promoted modularity and makes it easier to reuse code, add, replace or compare models without rewriting training logic.

2.5.3 Possible improvements

There are many approaches that could improve detection performance. One possibility could be introducing new complex architectures such as anchor-based detection which could provide better bounding box predictions by predicting offsets relative to predefined anchor shapes rather than relying only on cell-center predictions.

Another possible improvement could be to use more varied data augmentation during training. This could make the model more robust to variation in digit position, size, rotation and background noise, which could ultimately help reduce the number of false negatives and false positives.

Further hyperparameter tuning could improve training performance. For example, testing a wider range of learning rates, dropout rates, or tuning bounding box weights (λ_{bbox}) could lead to better detection accuracy.

Division of labor

Markus D. Simonsen focused on the object localization part (Section 1), including model implementation, training pipeline, and evaluation. Andreas W. Sløgedal focused on object detection part (Section 2), including model implementation, training pipeline and evaluation. Both of us contributed to the overall report writing and reviewed each other's work.

3 Table of Figures

Figure 1 - Example pictures with true bounding boxes.	3
Figure 2 - Class and no object distribution.....	4
Figure 3 - Height of bounding boxes by class.	4
Figure 4 - Width of bounding boxes by class.	5
Figure 5 - Model validation loss per. lambda.	10
Figure 6 - Model prediction on training set.	11
Figure 7 - Model predictions on validations set.	11
Figure 8 – Localization model predictions on the test set.	12
Figure 9 - Classification accuracy per digit.....	12
Figure 10 - Normalized confusion matrix.	13
Figure 11 - Normalized presence confusion matrix.....	14
Figure 12 - Localization IoU per digit.	14
Figure 13 - Localization center bias per digit.....	15
Figure 14 - Dataset (list_y_true_XXX.pt) analysis for object detection	17
Figure 15 - Loss analysis.....	22
Figure 16 - Detections on the training set.	23
Figure 17 - Detections on the validation set.	23

4 Bibliography

Prince, S. J. (2025). *Understanding Deep Learning* (online ed.). MIT Press.

5 Note on use of Artificial Intelligence

Following the faculties guidelines, artificial intelligence tools were used as support for this project. All decisions, implementations and experiments were conducted and verified by the authors.

B: As Support for Language Editing and Improvement of Text

ChatGPT 5.2 (OpenAI) was used to fix text the grammar, readability and clarify sections of the report. In addition, the tool was used to shorten and tighten the wording. No conclusions were generated by the tool. All provided content and rephrasing's were reviewed and edited before inclusion.

E: As Support for Programming

ChatGPT 5.2 (OpenAI) and Claude Opus 4.5 (Anthropic) were used as a programming assistant during the development of the project code. The systems were used for code explanations, debugging and code suggestions. None of the tools were used to generate complete solutions, all code was written, tested and validated by the authors.